

Punto 3.6 — Agrupaciones con proyección

Contexto del proyecto

Este checkpoint continúa M03/PROYECTO/3.5. Se mantiene la separación Domain/Application/Infrastructure/Console y el esquema se obtiene con migraciones, no con `EnsureCreated()`.

Objetivo práctico

Implementar y ejecutar **Agrupaciones con proyección** sobre `AceriaData`, inspeccionando resultados reales y, cuando corresponde, el SQL generado por EF Core 8.

Paso 1: Abrir el checkpoint

```
cd M03/PROYECTO/3.6
dotnet restore AceriaData.sln
```

La solución contiene los cuatro proyectos y puede abrirse directamente en Visual Studio 2022/2026 compatible con .NET 8.

Paso 2: Comprobar la historia de migraciones

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

El listado debe conservar las migraciones de M1 y M2, incluida `M2_2_12_Architecture`. LINQ y las estrategias de carga de M3 no cambian el esquema, por lo que M3 no inventa migraciones vacías.

Paso 3: Revisar los cambios de este punto

Los elementos trazados en este estado son: `ObtenerResumenPorClienteConOrdenes`, `ObtenerResumenPorClienteYEstado`, `ObtenerResumenPorClienteYEstadoConFiltro`, `ObtenerResumenMensualConOrdenes`, `ObtenerSqlAgrupacionClienteEstado`. El contrato crece de forma acumulativa, salvo en 3.12, donde el seam docente `IQueryable` se retira del puerto público y la composición queda encapsulada en `Infrastructure`.

Paso 4: Implementar y estudiar el caso de uso

Archivo: `src/AceriaData.Application/AgrupacionesUseCase.cs`

```

using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class AgrupacionesUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public AgrupacionesUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== AGRUPACIONES CON PROYECCIÓN ===");
        var porCliente = _unidad.Ordenes.ObtenerResumenPorClienteConOrdenes();
        var porClienteEstado = _unidad.Ordenes.ObtenerResumenPorClienteYEstado();
        var having = _unidad.Ordenes.ObtenerResumenPorClienteYEstadoConFiltro();
        var mensual = _unidad.Ordenes.ObtenerResumenMensualConOrdenes();
        var norte = porCliente.Single(x => x.Cliente == "Constructora del Norte");
        if (norte.TotalOrdenes != 3 || norte.Ordenes.Count != 3 || porClienteEstado.Count != 4 ||
            having.Count != 1 || mensual.Count != 5) throw new InvalidOperationException("Agrupaciones
            inesperadas.");
        Console.WriteLine($"Norte: {norte.TotalOrdenes} órdenes | HAVING: {having.Count} grupo");
        Console.WriteLine(_unidad.Ordenes.ObtenerSqlAgrupacionClienteEstado());
    }
}

```

Explicación línea a línea del caso de uso 3.6

Línea 1: `using AceriaData.Application.Interfaces;` → Importa los puertos de Application, en especial IUnidadDeTrabajo e IOrdenRepositorio, que desacoplan el caso de uso de EF Core.

Línea 3: `namespace AceriaData.Application.UseCases;` → Declara el espacio de nombres de la capa a la que pertenece el archivo.

Línea 5: `public sealed class AgrupacionesUseCase` → Declara el caso de uso concreto que coordina la demostración del punto.

Línea 6: `{` → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 7: `private readonly IUnidadDeTrabajo _unidad;` → Guarda el puerto de unidad de trabajo que da acceso al repositorio sin depender de DbContext.

Línea 8: `public AgrupacionesUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;` → Recibe la unidad de trabajo mediante inyección de dependencias.

Línea 10: `public void Ejecutar()` → Define la operación docente que ejecutará el composition root.

Línea 11: `{` → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 12: `Console.WriteLine("=== AGRUPACIONES CON PROYECCIÓN ===");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Línea 13: `var porCliente = _unidad.Ordenes.ObtenerResumenPorClienteConOrdenes();` → Obtiene cabeceras agrupadas por cliente y las órdenes de cada grupo con la estrategia acotada implementada en Infrastructure.

Línea 14: `var porClienteEstado = _unidad.Ordenes.ObtenerResumenPorClienteYEstado();` → Agrupa por la clave compuesta Cliente/Estado y materializa los agregados de cada combinación.

Línea 15: `var having = _unidad.Ordenes.ObtenerResumenPorClienteYEstadoConFiltro();` → Aplica un filtro sobre el resultado agrupado para demostrar el patrón equivalente a HAVING.

Línea 16: `var mensual = _unidad.Ordenes.ObtenerResumenMensualConOrdenes();` → Agrupa por año y mes y compone los resúmenes mensuales con las órdenes pertenecientes a cada grupo.

Línea 17: `var norte = porCliente.Single(x => x.Cliente == "Constructora del Norte");` → Localiza el único resumen de Constructora del Norte para validar el número de órdenes y la colección interna proyectada.

Línea 18: `if (norte.TotalOrdenes != 3 || norte.Ordenes.Count != 3 || porClienteEstado.Count != 4 || having.Count != 1 || mensual.Count != 5) throw new InvalidOperationException("Agrupaciones inesperadas.");` → Comprueba una condición E2E y hace fallar la ejecución si los resultados no coinciden con los datos esperados.

Línea 19: `Console.WriteLine($"Norte: {norte.TotalOrdenes} órdenes | HAVING: {having.Count} grupo");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Línea 20: `Console.WriteLine(_unidad.Ordenes.ObtenerSqlAgrupacionClienteEstado());` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Línea 21: `}` → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.

Línea 22: `}` → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.

| Paso 5: Preparar y ejecutar el composition root

Archivo: `src/AceriaData.Console/Program.cs`

```
using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<AgrupacionesUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateOnBuild = true,
    ValidateScopes = true
});
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<AgrupacionesUseCase>();
useCase.Ejecutar();

Console.WriteLine("3.6 OK");
```

Explicación línea a línea de Program.cs 3.6

Línea 1: `using AceriaData.Application.UseCases;` → Importa los casos de uso de Application para poder registrarlos y resolverlos desde el composition root.

Línea 2: `using AceriaData.ConsoleApp;` → Importa DemoData, responsable de sembrar el conjunto determinista usado por las pruebas E2E del checkpoint.

Línea 3: `using AceriaData.Infrastructure;` → Importa la extensión AddAceriaInfrastructure, que registra DbContext, repositorio y unidad de trabajo.

Línea 4: `using AceriaData.Infrastructure.Persistence;` → Importa AceriaDbContext para preparar la base de demostración desde el composition root.

Línea 5: `using Microsoft.EntityFrameworkCore;` → Importa las extensiones de EF Core utilizadas por Migrate, ChangeTracker y las operaciones de consulta del checkpoint.

Línea 6: `using Microsoft.Extensions.Configuration;` → Importa ConfigurationBuilder y las extensiones necesarias para cargar appsettings.json y variables de entorno.

Línea 7: `using Microsoft.Extensions.DependencyInjection;` → Importa ServiceCollection, AddScoped, CreateScope y GetRequiredService para componer el grafo de dependencias.

Línea 9: `var configuration = new ConfigurationBuilder()` → Inicia la construcción de la configuración de la aplicación.

Línea 10: `.SetBasePath(AppContext.BaseDirectory)` → Fija como base el directorio de ejecución para localizar appsettings.json.

Línea 11: `.AddJsonFile("appsettings.json", optional: false)` → Carga la configuración JSON obligatoria que contiene la cadena de conexión.

Línea 12: `.AddEnvironmentVariables()` → Permite sobrescribir configuración mediante variables de entorno.

Línea 13: `.Build();` → Finaliza la construcción y devuelve la configuración inmutable que se usará para leer ConnectionStrings:AceriaDB.

Línea 15: `var connectionString = configuration.GetConnectionString("AceriaDB")` → Obtiene la cadena AceriaDB desde la configuración.

Línea 16: `?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");` → Detiene el arranque con un error explícito si no existe la cadena de conexión AceriaDB, evitando ejecutar el ejercicio con una configuración incompleta.

Línea 18: `var services = new ServiceCollection();` → Crea el contenedor de servicios del composition root.

Línea 19: `services.AddAceriaInfrastructure(connectionString);` → Registra DbContext, repositorio y unidad de trabajo de Infrastructure.

Línea 20: `services.AddScoped<AgrupacionesUseCase>();` → Registra el caso de uso con ciclo de vida Scoped para compartir el mismo ámbito que DbContext.

Línea 22: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → Construye el proveedor de servicios y lo declara con using para liberar correctamente los servicios IDisposable al terminar el proceso.

Línea 23: `{` → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 24: `ValidateOnBuild = true,` → Obliga al contenedor a comprobar durante la construcción que los servicios registrados pueden resolverse.

Línea 25: `ValidateScopes = true` → Activa la validación de ciclos de vida para detectar, por ejemplo, un servicio Scoped consumido incorrectamente desde un Singleton.

Línea 26: `});` → Cierra las opciones y completa la llamada que construye el proveedor de servicios.

Línea 27: `using var scope = provider.CreateScope();` → Crea el ámbito Scoped que compartirá AceriaDbContext, IUnidadDeTrabajo y el caso de uso durante esta ejecución.

Línea 29: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el DbContext para preparar la base de demostración.

Línea 30: `context.Database.EnsureDeleted();` → Elimina solo la base de demostración para que el E2E comience desde un estado reproducible; no crea el esquema.

Línea 31: `context.Database.Migrate();` → Aplica la historia real de migraciones heredada de M2 para crear o actualizar el esquema.

Línea 32: `DemoData.Seed(context);` → Inserta el conjunto determinista de órdenes, planchas, detalles y aleaciones usado por los E2E.

Línea 33: `context.ChangeTracker.Clear();` → Desacopla las entidades sembradas para que las consultas se materialicen de nuevo desde SQL.

Línea 35: `var useCase = scope.ServiceProvider.GetRequiredService<AgrupacionesUseCase>();` → Resuelve el caso de uso concreto registrado para este checkpoint.

Línea 36: `useCase.Ejecutar();` → Ejecuta la demostración y sus aserciones E2E.

Línea 38: `Console.WriteLine("3.6 OK");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Paso 6: Compilar el estado acumulativo

```
dotnet build AceriaData.sln --configuration Release
```

La compilación debe terminar sin errores. Application no referencia Entity Framework Core; todas las llamadas específicas de EF permanecen en Infrastructure o en el composition root.

Paso 7: Ejecutar el E2E contra SQL Server LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

El proceso elimina la base de demostración, aplica `Database.Migrate()`, inserta un dataset determinista, limpia el Change Tracker y ejecuta el caso de uso. El marcador final obligatorio es:

```
3.6 OK
```

Paso 8: Analizar resultado y SQL

El caso de uso contiene aserciones que hacen fallar el proceso si cambian los resultados esperados. Cuando el punto estudia traducción SQL, el repositorio devuelve `ToQueryString()` y la consola imprime exactamente la sentencia que EF Core 8 ha construido para SQL Server. No se usa una sentencia SQL ficticia como prueba.

En 3.6, la colección interna por cliente se resuelve con **dos consultas acotadas** y composición posterior en memoria. Esto evita enseñar que una proyección de colección dentro de `GroupBy` implique necesariamente una consulta adicional por grupo. El SQL de `GROUP BY` y `HAVING` se valida con una consulta agregada independiente.

Paso 9: Diagnosticar un error común

Error: No hay que asumir que una colección interna dentro de `GroupBy` implica automáticamente `N+1`. Se debe inspeccionar la consulta real y diseñar una estrategia acotada.

Para diagnosticarlo, compara el método del repositorio, la salida E2E y el SQL obtenido con `ToQueryString()` cuando esté disponible. No cambies simultáneamente datos, query y expectativa: modifica una sola variable para poder atribuir la causa.

Laboratorio adicional del punto 3.6

Diagnóstico técnico

Una colección interna dentro de `GroupBy` no implica automáticamente N+1. El checkpoint usa una consulta de cabeceras agrupadas y otra de filas proyectadas, y compone en memoria con un número fijo de consultas. `Where(g => g.Count() > 1)` debe observarse como `HAVING`.

Reto resuelto y verificación adicional

Comprueba tres órdenes para Constructora del Norte y un único grupo cliente-estado con más de una orden. `ObtenerSqlAgrupacionClienteEstado()` debe contener un `GROUP BY` real.

Errores comunes revisados

Error	Causa	Solución
Colección interna agrupada	Se presupuso una traducción SQL o N+1 sin medirla	Inspeccionar la traducción o usar la estrategia acotada de dos consultas de 3.6
Filtro colocado en la fase equivocada	<code>Where</code> antes de <code>GroupBy</code> filtra filas; <code>Where</code> sobre el grupo filtra grupos	Elegir conscientemente <code>WHERE</code> o <code>HAVING</code> según lo que se quiera filtrar
Proyección relacional de grupo no válida	Se intenta proyectar una columna que no forma parte de la clave ni de un agregado en un <code>GROUP BY SQL</code>	Usar la clave/agregados o cambiar el <i>shape</i> y comprobar su traducción
N+1 asumido en agrupaciones	Se atribuye N+1 a cualquier colección interna sin medir consultas	Inspeccionar la traducción real y usar una estrategia acotada como la de 3.6
SelectMany sin proyección	Se aplanan sin proyectar	Añadir una proyección clara
Agrupación anidada	Se presupuso una traducción o número de consultas fijo	Revisar el <i>shape</i> y hacer explícita la frontera cliente/servidor si hace falta

Analogía operativa

Las agrupaciones con proyección en una acería son como los informes agrupados que el jefe de planta pide al archivo central. En lugar de una lista plana de órdenes, el jefe pide un informe agrupado por cliente: para cada cliente, cuántas órdenes tiene y cuáles son. El archivo central agrupa las carpetas por cliente, cuenta las órdenes y prepara una lista con los números de cada orden. Proyectar una colección interna dentro de un grupo es como adjuntar la lista de órdenes al informe del cliente. El filtro `Having` es como pedir solo los clientes que tienen más de una orden. La agrupación por múltiples claves es como agrupar por cliente y estado a la vez. `SelectMany` es como aplanar los grupos en una lista plana. Así funcionan las agrupaciones con proyección en EF Core: se agrupan los datos en el servidor, se proyectan los resúmenes y se adjuntan las colecciones internas cuando es necesario. El resultado es un informe estructurado y eficiente.

Paso 10: Reto resuelto y conexión con el siguiente punto

Reto: Construye un `HAVING` para grupos con más de una orden y verifica que el SQL contiene `GROUP BY/HAVING`.

Cómo activar el reto en el checkpoint

Estos bloques ya existen comentados en el checkpoint. Actívalos de forma coordinada sólo para esta variante y restaura después el estado original.

Archivo: `src/AceriaData.Application/Interfaces.cs`

Marcador: RETO M03 3.6 - PUERTO SQL HAVING

```
string ObtenerSqlAgrupacionClienteEstadoConHavingReto();
```

Archivo: src/AceriaData.Infrastructure/Repositories/Repositories.cs

Marcador: RETO M03 3.6 - SQL GROUP BY + HAVING

```
public string ObtenerSqlAgrupacionClienteEstadoConHavingReto() => _context.OrdenesFabricacion
    .GroupBy(o => new { o.Cliente, o.Estado })
    .Where(g => g.Count() > 1)
    .Select(g => new { g.Key.Cliente, g.Key.Estado, Total = g.Count() })
    .ToString();
```

Archivo: src/AceriaData.Application/AgrupacionesUseCase.cs

Marcador: RETO M03 3.6 - HAVING PARA GRUPOS CON MAS DE UNA ORDEN

```
// Demuestra el Paso 10 con el único grupo cliente/estado repetido del dataset.
var gruposReto = _unidad.Ordenes.ObtenerResumenPorClienteYEstadoConFiltro();

if (gruposReto.Count != 1 ||
    gruposReto[0].Cliente != "Constructora del Norte" ||
    gruposReto[0].Estado != "Pendiente" ||
    gruposReto[0].TotalOrdenes != 2)
    throw new InvalidOperationException("Reto 3.6: el grupo HAVING no coincide con el
dataset.");

var sqlHavingReto = _unidad.Ordenes.ObtenerSqlAgrupacionClienteEstadoConHavingReto();

Console.WriteLine($"Reto 3.6 OK | {gruposReto[0].Cliente} | {gruposReto[0].Estado} | Órdenes:
{gruposReto[0].TotalOrdenes}");
Console.WriteLine("RETO_HAVING_SQL_INICIO");
Console.WriteLine(sqlHavingReto);
Console.WriteLine("RETO_HAVING_SQL_FIN");
```

Comprobación esperada: GROUP BY con HAVING real para conservar sólo grupos con más de una orden.

La solución está representada por el estado acumulativo del propio checkpoint y sus métodos de repositorio. Tras validar **3.6 OK**, el siguiente estado parte exactamente de esta solución y añade **Joins y navegación en consultas**.